Data 188 Introduction to Deep Learning
Spring 2026 Eric Kim

Discussion 09

This week we continue our study of transformers by focusing on the encoder-decoder setting. We will practice cross-attention (differing from self-attention from last week) tensor shapes, keep track of the tensors inside a decoder block, and reason about why attention is nonlinear, data-dependent, and order-sensitive.

1. Cross-Attention Shapes

Recall that in multi-head cross-attention, the queries come from the target sequence, while the keys and values come from the source sequence. For head $h \in \{1, \ldots, H\}$,

$$Q_h = X_{\text{tgt}} W_h^{(q)}, \qquad K_h = X_{\text{src}} W_h^{(k)}, \qquad V_h = X_{\text{src}} W_h^{(v)},$$

$$\mathrm{H}_h = \mathrm{Softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_q}}\right) V_h,$$

and the full multi-head output is

$$Y = \mathrm{Concat}(\mathrm{H}_1, \ldots, \mathrm{H}_H)\, W^{(o)}.$$

In the figure below, we will focus on a *single* cross-attention block (that is, one head). As a result, each target token computes attention weights over the source tokens, and these weights depend on the particular input sequences in the batch.

Assume

$$X_{\text{tgt}} \in \mathbb{R}^{B \times n_{\text{tgt}} \times d}, \qquad X_{\text{src}} \in \mathbb{R}^{B \times n_{\text{src}} \times d},$$

and assume the query/key dimension is d_q (so $d_k = d_q$) and the value dimension is d_v.

 All Rights Reserved. This may not be publicly shared without explicit permission.

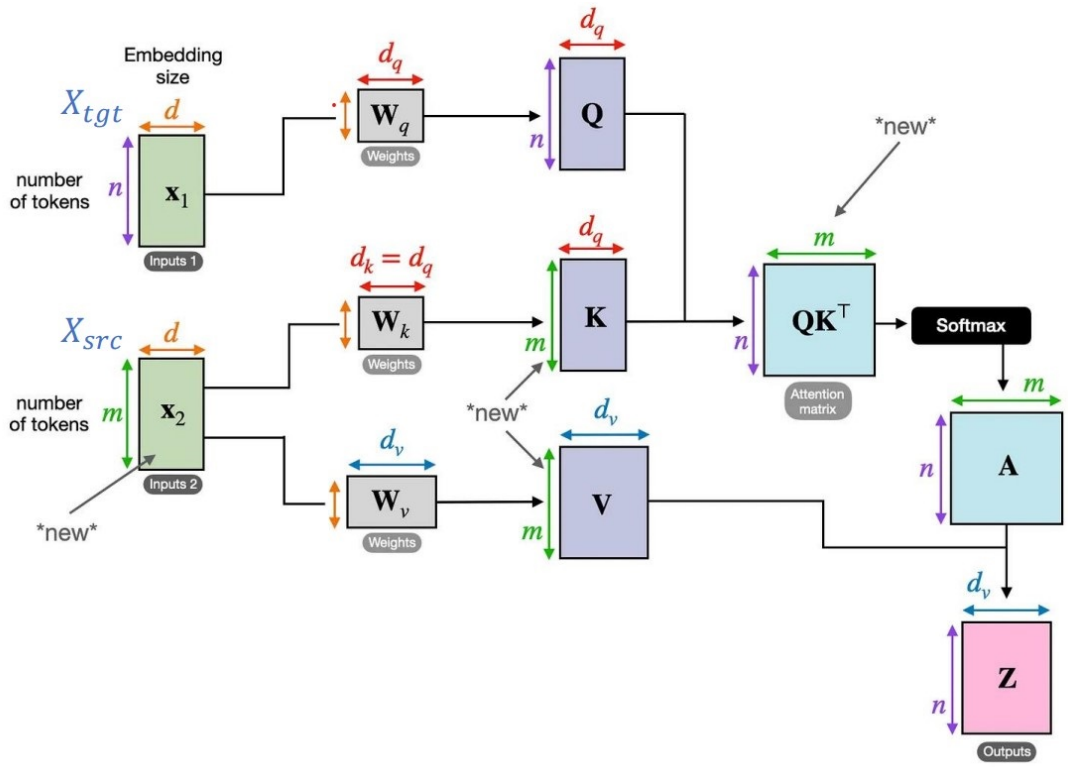

Figure 1: Cross-attention block.

You may annotate the figure as you work, but please write your final answers below.

(a) What are the shapes of the projection weight matrices $W^{(q)}$, $W^{(k)}$, and $W^{(v)}$?

Solution: Since each token starts in the model embedding space of dimension d, the projection matrices map from d to the corresponding query/key/value dimensions:

$$W^{(q)} \in \mathbb{R}^{d \times d_q}, \qquad W^{(k)} \in \mathbb{R}^{d \times d_q}, \qquad W^{(v)} \in \mathbb{R}^{d \times d_v}.$$

(b) What are the shapes of the projected activations Q, K, and V?

Solution: The target sequence produces the queries, and the source sequence produces the keys and values:

$$Q = X_{\text{tgt}} W^{(q)} \in \mathbb{R}^{B \times n_{\text{tgt}} \times d_q},$$

$$K = X_{\text{src}} W^{(k)} \in \mathbb{R}^{B \times n_{\text{src}} \times d_q}, \qquad V = X_{\text{src}} W^{(v)} \in \mathbb{R}^{B \times n_{\text{src}} \times d_v}.$$

(c) What are the shapes of the score matrix $QK^\top$, the attention matrix

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_q}}\right),$$

and the final output

$$Z = AV?$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

Then, in one or two sentences, explain what the entries of A represent. Why can A be viewed as a set of *data-dependent* weights?

Solution: For each batch element, each target token compares itself against every source token, so

$$QK^\top \in \mathbb{R}^{B \times n_{\text{tgt}} \times n_{\text{src}}}.$$

The scaling and softmax do not change shape, so

$$A \in \mathbb{R}^{B \times n_{\text{tgt}} \times n_{\text{src}}}.$$

Finally, multiplying by V collapses the source-token dimension and leaves one output vector per target token:

$$Z = AV \in \mathbb{R}^{B \times n_{\text{tgt}} \times d_v}.$$

Each row of A gives a distribution of attention weights over the n_{src} source positions for one target position. In particular, A_{ij} tells us how much target token i attends to source token j.

These weights are *data-dependent* because they are not fixed after training like the weights of a standard linear layer. Instead, they are computed from the current input sequences through $QK^\top$, so changing X_{tgt} or X_{src} changes A.

2. Decoder Block Shapes in an Encoder-Decoder Transformer

The figure below zooms in on one decoder block inside an encoder-decoder transformer for machine translation.

Assume the encoder has already processed the source sequence and produced

$$E \in \mathbb{R}^{B \times n_{\text{src}} \times d}.$$

The decoder takes target-side hidden states

$$X \in \mathbb{R}^{B \times n_{\text{tgt}} \times d}$$

as input to this block.

For both attention sublayers, assume query/key dimension d_q (so $d_k = d_q$), value dimension d_v, and an output projection back to model dimension d. For the MLP, assume the hidden width is d_{ff}. LayerNorm and residual connections do not change tensor shapes.

 All Rights Reserved. This may not be publicly shared without explicit permission.

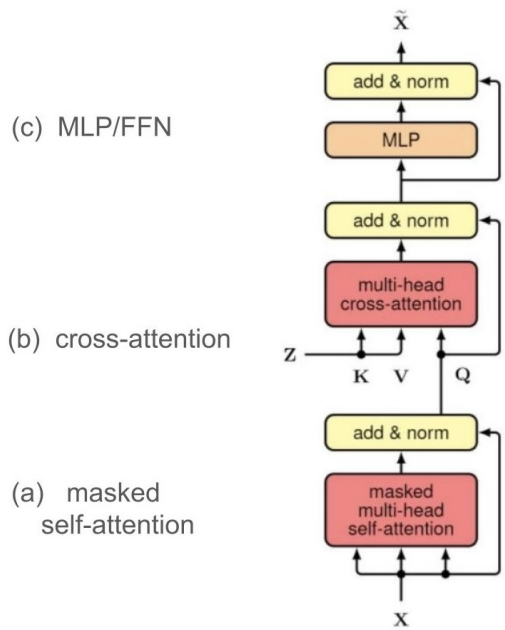

Figure 2: One decoder block in an encoder-decoder transformer.

You may annotate the figure as you work, but please write your final answers below.

(a) In the masked self-attention sublayer, what are the shapes of

$$W_{\text{self}}^{(q)},\ W_{\text{self}}^{(k)},\ W_{\text{self}}^{(v)},\ W_{\text{self}}^{(o)},$$

the activations

$$Q_{\text{self}},\ K_{\text{self}},\ V_{\text{self}},$$

the masked attention matrix A_{self}, and the projected sublayer output O_{self} right before the first add & norm?

Solution: Since masked self-attention uses the decoder sequence for queries, keys, and values:

$$W_{\text{self}}^{(q)} \in \mathbb{R}^{d\times d_q}, \qquad W_{\text{self}}^{(k)} \in \mathbb{R}^{d\times d_q}, \qquad W_{\text{self}}^{(v)} \in \mathbb{R}^{d\times d_v},$$

$$W_{\text{self}}^{(o)} \in \mathbb{R}^{d_v\times d}.$$

The projected activations are

$$Q_{\text{self}}, K_{\text{self}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times d_q}, \qquad V_{\text{self}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times d_v}.$$

Each target position attends over target positions, so

$$A_{\text{self}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times n_{\text{tgt}}}.$$

The pre-output-projection attention result has shape $\mathbb{R}^{B\times n_{\text{tgt}}\times d_v}$, and after applying $W_{\text{self}}^{(o)}$ we get

$$O_{\text{self}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times d}.$$

This must match the residual input shape so the add & norm can be applied.

 All Rights Reserved. This may not be publicly shared without explicit permission.

(b) In the cross-attention sublayer, the queries come from the decoder stream, while the keys and values come from the encoder output E. What are the shapes of

$$W_{\text{cross}}^{(q)},\ W_{\text{cross}}^{(k)},\ W_{\text{cross}}^{(v)},\ W_{\text{cross}}^{(o)},$$

the activations

$$Q_{\text{cross}},\ K_{\text{cross}},\ V_{\text{cross}},$$

the attention matrix A_{cross}, and the projected sublayer output O_{cross} right before the second add & norm?

Solution: The projection weight matrices have the same shapes as before:

$$W_{\text{cross}}^{(q)} \in \mathbb{R}^{d\times d_q}, \qquad W_{\text{cross}}^{(k)} \in \mathbb{R}^{d\times d_q}, \qquad W_{\text{cross}}^{(v)} \in \mathbb{R}^{d\times d_v},$$
$$W_{\text{cross}}^{(o)} \in \mathbb{R}^{d_v\times d}.$$

Queries come from the decoder stream, so

$$Q_{\text{cross}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times d_q}.$$

Keys and values come from the encoder output, so

$$K_{\text{cross}} \in \mathbb{R}^{B\times n_{\text{src}}\times d_q}, \qquad V_{\text{cross}} \in \mathbb{R}^{B\times n_{\text{src}}\times d_v}.$$

Therefore the cross-attention matrix has one row per target token and one column per source token:

$$A_{\text{cross}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times n_{\text{src}}}.$$

The pre-output-projection attention result has shape $\mathbb{R}^{B\times n_{\text{tgt}}\times d_v}$, and after applying $W_{\text{cross}}^{(o)}$ we get

$$O_{\text{cross}} \in \mathbb{R}^{B\times n_{\text{tgt}}\times d}.$$

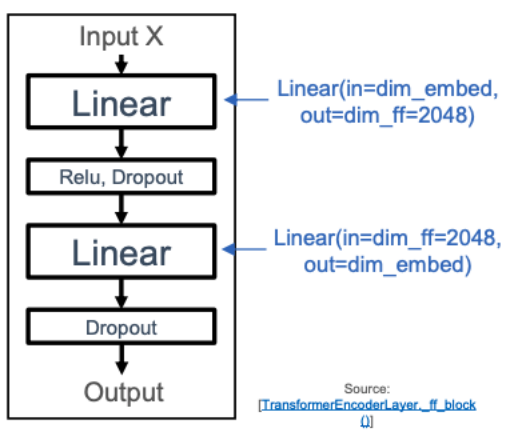

Figure 3: FNN Block (Lec 16 Slide 36).

(c) In the above **MLP/FFN sublayer**, the first linear layer expands from model width d to hidden width d_{ff}, a nonlinearity is applied, and the second linear layer projects back to width d.

 All Rights Reserved. This may not be publicly shared without explicit permission.

$$\text{hidden} = \text{ReLU}(XW_1)$$
$$\tilde{X} = \text{hidden}\, W_2$$

What are the shapes of the two MLP weight matrices, the hidden activation, and the final decoder-block output $\tilde{X}$?

Solution: The first linear layer expands the last dimension:

$$W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}, \qquad XW_1 \in \mathbb{R}^{B \times n_{\text{tgt}} \times d_{\text{ff}}}.$$

The nonlinearity does not change shape, so the hidden activation is still

$$\mathbb{R}^{B \times n_{\text{tgt}} \times d_{\text{ff}}}.$$

The second linear layer projects back to the model width:

$$W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}.$$

Therefore the MLP output, and hence the final output of the decoder block after the last add & norm (after the FFN block), is

$$\tilde{X} \in \mathbb{R}^{B \times n_{\text{tgt}} \times d}.$$

3. Why Attention Is More Than a Linear Layer

Recall the equations for a single attention head:

$$\begin{aligned} Q &= XW^{(q)} \\ K &= XW^{(k)} \\ V &= XW^{(v)} \\ H &= \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_q}}\right) V. \end{aligned}$$

In this question, ignore batching and assume $X \in \mathbb{R}^{n \times d}$, where n is the sequence length.

(a) For this part only, ignore the softmax and normalization ($\sqrt{d_q}$) and set

$$W^{(q)} = W^{(k)} = W^{(v)} = I.$$

Show that the attention output simplifies to

$$H = (XX^\top)X.$$

Solution: Under these assumptions we have

$$Q = X, \qquad K = X, \qquad V = X.$$

Ignoring the softmax gives

$$H = (QK^\top)V = (XX^\top)X.$$

 All Rights Reserved. This may not be publicly shared without explicit permission.

(b) Why does the expression $(XX^\top)X$ show that attention is *nonlinear* in X? Why does it also show that attention *mixes information across tokens*?

Solution: The matrix $XX^\top$ contains all pairwise dot products between tokens:

$$(XX^\top)_{ij} = x_i^\top x_j,$$

where x_i is the ith row of X.

This already shows token mixing: entry (i, j) measures how strongly token i interacts with token j, and the product $(XX^\top)X$ uses those interaction strengths to form new token representations from *all* rows of X.

It is also nonlinear in X. A standard linear map has the form XW and is linear in the entries of X. In contrast, $XX^\top$ contains products of entries of X, and the full expression $(XX^\top)X$ contains higher-order products of entries of X. So the mapping $X \mapsto (XX^\top)X$ is not linear.

(c) Now restore the learned projections and the softmax. Rewrite the attention output in the form

$$H = A(X)XW^{(v)},$$

where $A(X)$ is an $n \times n$ matrix that depends on the current input.

In what sense can attention be viewed as a *data-dependent* linear layer? How is this different from an ordinary linear layer XW?

Solution: We can write

$$\begin{aligned} H &= \mathrm{Softmax}\left(\frac{QK^\top}{\sqrt{d_q}}\right)V \\ &= \mathrm{Softmax}\left(\frac{XW^{(q)}(XW^{(k)})^\top}{\sqrt{d_q}}\right)XW^{(v)} \\ &= \mathrm{Softmax}\left(\frac{XW^{(q)}W^{(k)\top}X^\top}{\sqrt{d_q}}\right)XW^{(v)}. \end{aligned}$$

So if we define

$$A(X) = \mathrm{Softmax}\left(\frac{XW^{(q)}W^{(k)\top}X^\top}{\sqrt{d_q}}\right),$$

then

$$H = A(X)XW^{(v)}.$$

This looks like a linear transformation applied to X, except the token-mixing matrix $A(X)$ is not fixed in advance: it is computed from the current input sequence. So different inputs induce different effective weights.

By contrast, an ordinary linear layer applies the same weight matrix W to every example:

$$X \mapsto XW.$$

The weights do not depend on the input, and there is no analogous data-dependent token-to-token interaction matrix.

 All Rights Reserved. This may not be publicly shared without explicit permission.

4. Why the Decoder Looks the Way It Does

This final question focuses on several design choices that appear in encoder-only, decoder-only, and encoder-decoder transformers.

(a) Suppose we remove positional encodings entirely from a transformer. What kind of information would the model lose? Give one concrete example of two sentences that would become difficult to distinguish based on meaning.

Solution: Without positional encodings, the model has no direct way to tell which token came first, second, third, and so on. In other words, it loses word-order information.

For example, the sentences

```
the cat chased the mouse
```

and

```
the mouse chased the cat
```

contain the same tokens but have different meanings because of their order. Without positional information, a transformer would struggle to represent that distinction.

(b) Give one common use case for each transformer variant below.

i. encoder-only
ii. decoder-only
iii. encoder-decoder

Solution: Typical examples are:

i. encoder-only: building strong token or sequence representations for tasks such as classification or tagging,
ii. decoder-only: autoregressive generation tasks such as next-token prediction, chat, or text completion (ChatGPT is an example),
iii. encoder-decoder: sequence-to-sequence tasks such as machine translation or summarization.

(c) In the decoder, why do we apply a *causal mask* in the self-attention block for tasks such as machine translation?

Solution: The decoder predicts the target sequence one token at a time. When predicting token t, it should only be allowed to use earlier target tokens, not future ones.

The causal mask prevents the model from *looking ahead*. Without it, during training the decoder could directly attend to the correct future target tokens and learn to rely on information that will not be available at inference time.

(d) Do we also apply a *causal mask* in the cross-attention block of an encoder-decoder transformer for machine translation? Why or why not?

Solution: No. In cross-attention, the decoder queries the *encoder output*, which represents the full source sentence. For machine translation, when generating a target token, we generally want the decoder to be able to attend to *any* source position.

So cross-attention does not use a causal mask over source positions. However, we may still use a *padding mask* so that the decoder ignores padded source tokens.

Contributors:

- Eric Kim.
- Terry Kim.

 All Rights Reserved. This may not be publicly shared without explicit permission.